Practical course

Compiler and Hardware Co-Design

(WiSe 2026/27)

Part 1: Compilers

System Research Group

https://dse.in.tum.de/

Today

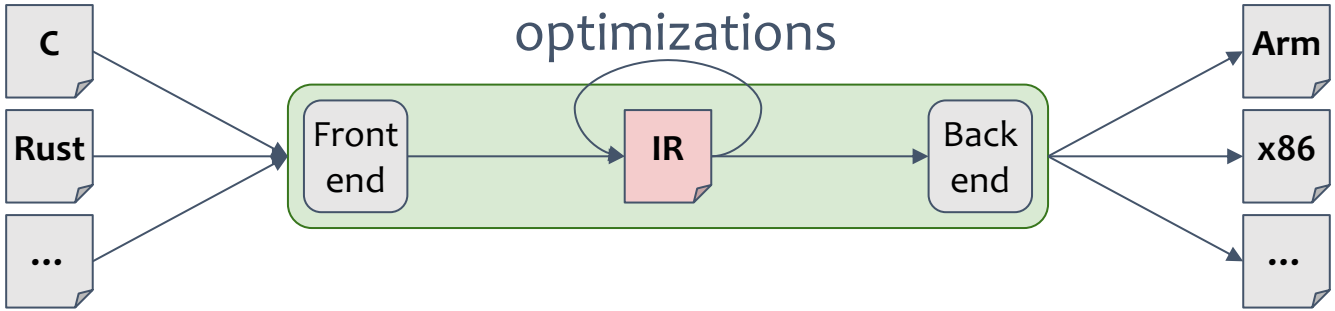
Compilers

- Quick introduction to MLIR
  - Classic compiler pipelines
  - Intermediate representations
  - MLIR: operations, types, dialects, lowering
- Working with MLIR in C++
  - Passes, walking, building and replacing IR
- The first task
  - Intuition
  - Task

Classic compiler pipelines



Classic compiler pipelines



- Frontend: parsing & lowering
- Backend: machine code generation

Optimizations: make your code fast

Intermediate representations (IR)

Generic abstraction to do optimizations on

- No concrete registers: infinite virtual registers (**values**)
- No architecture specific instructions: generic **operations**

Example: LLVM IR

```
int f(int x) {
    int y = x + 1;
    int z = x * 42;

    return y;
}
```

```
define i32 @f(i32 %x)
{
  %y = add i32 %x, 1
  %z = mul i32 %x, 42
  ret i32 %y
}
```

Values (written once)

Operations

Intermediate representations (IR)

Problem: What if the abstraction is not right?

- No structured control flow (if/else, for/while)
- Opaque pointers

```
void matmul(int A[2][2],int B[2][2],
            int C[2][2])
{
  for (int i = 0; i < 2; i++)
    for (int j = 0; j < 2; j++) {
      C[i][j] = 0;
      for (int k = 0; k < 2; k++)
        C[i][j] += A[i][k] * B[k][j];
    }
}
```

- Flat loops (needs analysis)
- Lost memory shape

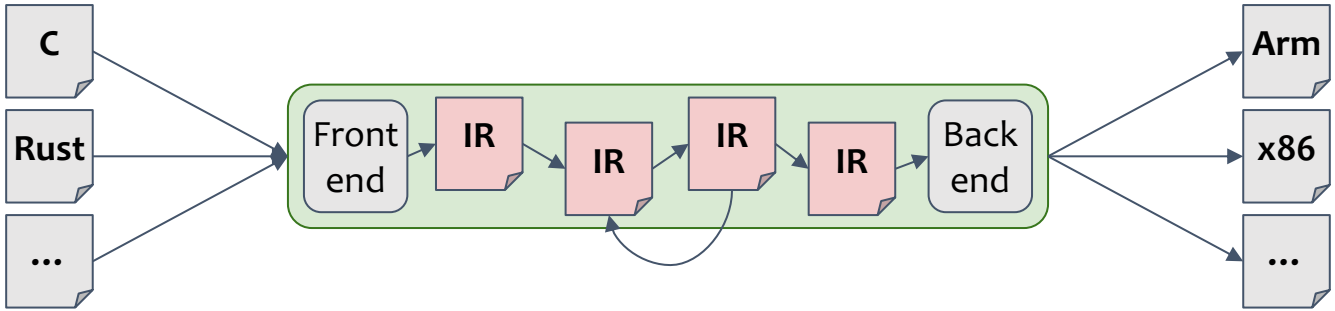
```
define void @matmul(ptr %A,ptr %B,ptr %C)
{
  i.loop: ...
  j.loop: ...
  k.loop: ...
  << load, mul, sum, store >>
  %k.next = add i64 %k, 1
  %k.done = icmp eq i64 %k.next, 2
  br i1 %k.done,label %k.end,label %k.loop

k.end:
  ...
```

MLIR

Template IR

- Allows creation of interoperable IRs
- No IR provides your abstraction? Make your own!

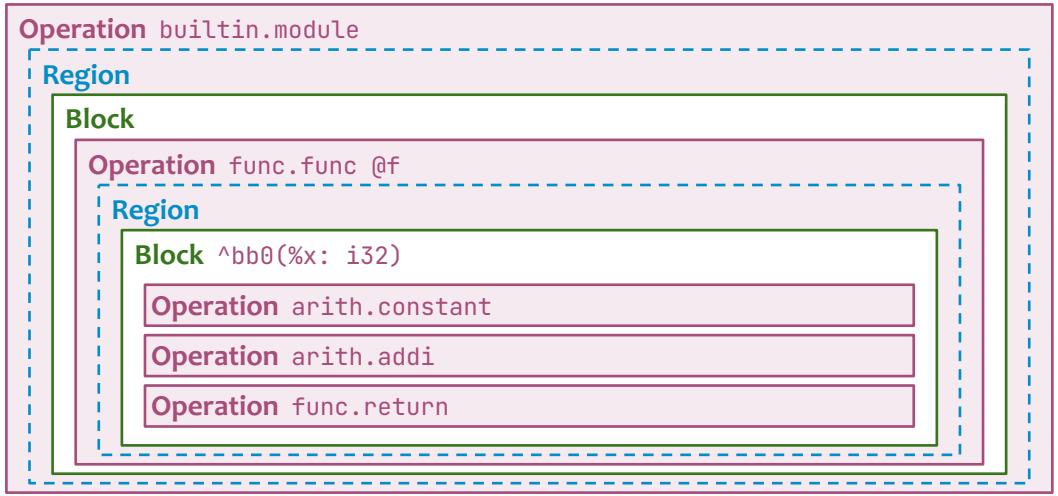


MLIR – Structure

Everything is an operation

- Operation → Region(s) → Block(s) → Operations → …
- `builtin.module` and `func.func` are operations, too

```
builtin.module {
  func.func @f(%x: i32) -> i32 {
    %c1 = arith.constant 1 : i32
    %y = arith.addi %x, %c1 : i32
    func.return %y : i32
  }
}
```

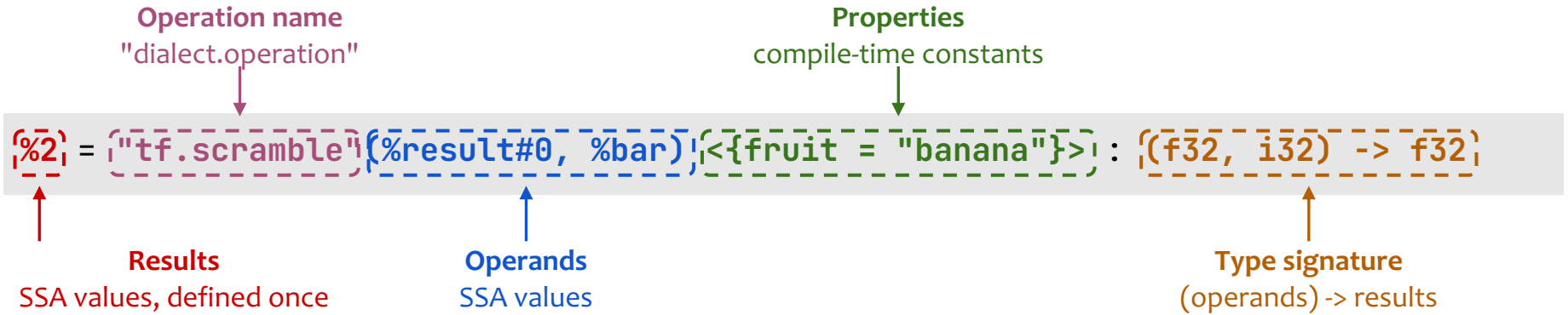


LLVM IR: fixed levels – Module → Function → Basic block → Instruction

MLIR: any op can hold regions, so nesting is open-ended

MLIR – Operations

Generic form: every operation has the same shape



- Optional: attribute dictionary `{…}`, regions `({…})`, successor blocks, location `loc(…)`
- MLIR doesn't need to know an op to parse it: `tf.scramble` works with `--allow-unregistered-dialect`

MLIR – Custom syntax

Dialects can define a readable syntax for their ops

- `mlir-opt --mlir-print-op-generic` prints the generic form

Custom syntax (easier to read)

```
func.func @f(%x: i32) -> i32 {
  %c1 = arith.constant 1 : i32
  %y = arith.addi %x, %c1 : i32
  return %y : i32
}
```

`return` is short for `func.return` inside a function

Generic syntax

```
"func.func"() <{function_type = (i32) -> i32,
                sym_name = "f"}> ({
^bb0(%arg0: i32):
  %0 = "arith.constant"() <{value = 1 : i32}>
       : () -> i32
  %1 = "arith.addi"(%arg0, %0)
       <{overflowFlags = #arith.overflow<none>}>
       : (i32, i32) -> i32
  "func.return"(%1) : (i32) -> ()
}) : () -> ()
```

The body is a **region** (`{ … }`); `^bb0` is its entry **block**, `%arg0` a block argument

MLIR – Types and attributes

Types: what kind of value it is

```
i1, i32, i64, index          // integers
f16, f32, f64                // floats
memref<4x3xf32>              // buffer with a shape
memref<?x?xf32, strided<[?, ?], offset: ?>>
tensor<?x4xf32>              // immutable value
vector<8xf32>                // SIMD vector
(i32, i32) -> i32            // function type
```

`?` = size only known at runtime

Attributes: compile-time constants

```
42 : i32                     // integer
1.0 : f32                    // float
"banana"                     // string
@external_matmul             // symbol reference
[1, 2, 3]                    // array
affine_map<(i, j) -> (j, i)>
```

In C++:

uniqued in the `MLIRContext`

```
Type f32 = Float32Type::get(ctx);          // f32
auto A = MemRefType::get({4, 3}, f32);     // memref<4x3xf32>
Attribute c = b.getI32IntegerAttr(42);     // 42 : i32
```

MLIR – Dialects

A dialect = ops, types and attributes for one level of abstraction

high level

`linalg` `tensor`	whole computations: `linalg.matmul`, `linalg.conv_2d`, …
`affine`	loops and indices with affine bounds: easy to analyze
`scf` `memref` `arith`	structured loops and ifs, buffers, scalar math
`cf`	blocks and branches – like LLVM IR
`llvm`	LLVM IR in MLIR, exported with `mlir-translate`

low level

task input

task output

Used everywhere: `builtin` (module, types) and `func` (functions, calls)

MLIR

Dialects (examples)

- affine: each affine loop is one operation
- memref: memory references (types) that remember their shape

```
void matmul(float A[4][3], float B[3][5],
            float C[4][5])
{
  for (int i = 0; i < 4; i++)
    for (int j = 0; j < 5; j++)
      for (int k = 0; k < 3; k++)
        C[i][j] += A[i][k] * B[k][j];
}
```

```
func.func @matmul(%A: memref<4x3xf32>,
                  %B: memref<3x5xf32>,
                  %C: memref<4x5xf32>) {
 affine.for %i = 0 to 4 {
  affine.for %j = 0 to 5 {
   affine.for %k = 0 to 3 {
    %a = affine.load %A[%i, %k] : memref<4x3xf32>
    %b = affine.load %B[%k, %j] : memref<3x5xf32>
    %p = arith.mulf %a, %b : f32
    %c = affine.load %C[%i, %j] : memref<4x5xf32>
    %s = arith.addf %c, %p : f32
    affine.store %s, %C[%i, %j] : memref<4x5xf32>
   } } }
```

MLIR – Structured control flow

if/else in MLIR

- `scf.if` keeps the structure and returns a value via `scf.yield`
- `cf` = blocks and branches; block arguments replace `phi`

scf: structured

```
%c0 = arith.constant 0 : i32
%cmp = arith.cmpi sgt, %cond, %c0 : i32
%r = scf.if %cmp -> i32 {
  %c1 = arith.constant 1 : i32
  %a = arith.addi %val, %c1 : i32
  scf.yield %a : i32
} else {
  %c42 = arith.constant 42 : i32
  %b = arith.addi %val, %c42 : i32
  scf.yield %b : i32
}
```

result of `scf.if`

cf: after `--convert-scf-to-cf`

```
  %c0 = arith.constant 0 : i32
  %cmp = arith.cmpi sgt, %cond, %c0 : i32
  cf.cond_br %cmp, ^then, ^else
^then:
  %c1 = arith.constant 1 : i32
  %a = arith.addi %val, %c1 : i32
  cf.br ^end(%a : i32)
^else:
  %c42 = arith.constant 42 : i32
  %b = arith.addi %val, %c42 : i32
  cf.br ^end(%b : i32)
^end(%r: i32):
```

≙ `phi` in LLVM IR

MLIR – Progressive lowering

Lower step by step: each step is one pass, dialects can be mixed

```
func.func @matmul(%A: memref<4x3xf32>, %B: memref<3x5xf32>,
                  %C: memref<4x5xf32>) {
  linalg.matmul ins(%A, %B : memref<4x3xf32>, memref<3x5xf32>)
                outs(%C : memref<4x5xf32>)
  return
}
```

`--convert-linalg-to-affine-loops` turns this into exactly the `affine.for` nest from before




```
$ mlir-opt matmul.mlir --convert-linalg-to-affine-loops --lower-affine \
    --convert-scf-to-cf --finalize-memref-to-llvm --convert-arith-to-llvm \
    --convert-func-to-llvm --convert-cf-to-llvm --reconcile-unrealized-casts \
  | mlir-translate --mlir-to-llvmir > matmul.ll
```

In LLVM IR, every `memref` argument becomes pointers + offset + sizes + strides

MLIR – Tools

`mlir-opt`: like LLVM's `opt` – parse, run passes, verify, print

- The verifier runs after every pass: broken IR is reported right away
- Try it online: Compiler Explorer (godbolt.org) supports MLIR

```
$ mlir-opt in.mlir --canonicalize --cse           # run passes
$ mlir-opt in.mlir --pass-pipeline="builtin.module(func.func(canonicalize))"
$ mlir-opt in.mlir --mlir-print-ir-after-all      # IR after every pass
$ mlir-opt in.mlir --mlir-print-op-generic        # generic syntax
$ mlir-opt in.mlir --allow-unregistered-dialect   # accept unknown ops
$ mlir-opt --help | grep linalg                   # find passes
$ mlir-translate out.mlir --mlir-to-llvmir        # llvm dialect -> LLVM IR
```

```
$ mlir-opt matmul.mlir --lower-affine --mlir-print-ir-after-all
// -----// IR Dump After LowerAffinePass (lower-affine) //----- //
module {  ...
```

MLIR in C++ – Core classes

- `MLIRContext`: owns dialects, types, attributes
- `Operation *`: any op – `getName()`, `getOperands()`, `getResults()`, `getRegions()`
- `arith::AddFOp`, `affine::AffineForOp`, …: typed op, a thin wrapper around `Operation *` with named accessors (pass by value)
- `Value`: op result or block argument
- `Block`, `Region`: containers
- `Location`: source position, needed to create ops

```
if (auto add = dyn_cast<arith::AddFOp>(op)) {
  Value lhs = add.getLhs();               // named accessor
  Operation *def = lhs.getDefiningOp();   // null for block args
  Type t = add.getType();                 // f32
}
if (isa<affine::AffineForOp>(op)) { /* ... */ }

Operation *raw = loop.getOperation();     // -> Operation*
Block *body = loop.getBody();             // the loop body
Value iv = loop.getInductionVar();        // %i
```

LLVM	MLIR
`Instruction *`	`Operation *`
`BinaryOperator`	`arith::AddIOp`
`dyn_cast<LoadInst>(I)`	`dyn_cast<affine::AffineLoadOp>(op)`
`Value *`	`Value`
`BasicBlock, Function`	`Block, Region, func::FuncOp`

MLIR in C++ – Hello world pass

Print the number of operations in every function

```
struct HelloPass
    : PassWrapper<HelloPass, OperationPass<ModuleOp>> {
  MLIR_DEFINE_EXPLICIT_INTERNAL_INLINE_TYPE_ID(HelloPass)
  StringRef getArgument() const final { return "hello"; }

  void runOnOperation() override {
    getOperation().walk([](func::FuncOp func) {
      int numOps = 0;
      func.walk([&](Operation *op) { numOps++; });
      llvm::errs() << func.getName() << ": "
                   << numOps << " ops\n";
    });
  }
};
```

Runs on the whole module

Name used in `--pass-pipeline`

`walk()` visits all nested ops of the callback's type

`Operation *` matches every op

MLIR in C++ – Building and running

Your pass is a plugin for `mlir-opt` – no need to build MLIR yourself

```
extern "C" LLVM_ATTRIBUTE_WEAK ::mlir::PassPluginLibraryInfo
mlirGetPassPluginInfo() {
  return {MLIR_PLUGIN_API_VERSION, "Hello", "0.1.0",
          []() { PassRegistration<HelloPass>(); }};
}
```

Plugin entry point: registers the pass

```
add_llvm_library(Hello MODULE Hello.cpp PLUGIN_TOOL mlir-opt)
```

CMake: build a shared library

```
$ cmake -B build -G Ninja && cmake --build build
$ mlir-opt avg.mlir --load-pass-plugin=build/src/Hello.so \
    --pass-pipeline="builtin.module(hello)"
avg: 5 ops
module { ...
```

Run plugin passes via `--pass-pipeline`

Task: `MatMulOpt.so`, pass `matmul-opt`

MLIR in C++ – Walking and matching

MLIR's visitor pattern: `walk()` with a typed callback

```
module.walk([](affine::AffineForOp loop) {   // only affine.for ops
  int n = 0;
  for (Operation &op : loop.getBody()->without_terminator())
    n++;
  bool outermost = !loop->getParentOfType<affine::AffineForOp>();
  llvm::errs() << "loop: " << n << " ops in body"
               << (outermost ? " (outermost)" : "") << "\n";
});

module.walk([](arith::AddFOp add) {
  if (add.getRhs().getDefiningOp<arith::MulFOp>())
    add.emitRemark("multiply-add");
});
```

Also useful: `value.getUsers()`, `op->getParentOp()`, `WalkResult::interrupt()`

Output for the matmul:

```
loop: 6 ops in body
loop: 1 ops in body
loop: 1 ops in body (outermost)
matmul.mlir:10:16: remark:
    multiply-add
```

Post-order: inner loops are visited first

`without_terminator()` skips the implicit `affine.yield`

`getDefiningOp<T>()`: from a value to the op that produced it

MLIR in C++ – Building IR

`OpBuilder` ≈ LLVM's `IRBuilder`: set an insertion point, then create ops

LLVM

```
IRBuilder<> Builder(&I);
Value *Sum = Builder.CreateAdd(X, Y);
I.replaceAllUsesWith(Sum);
I.eraseFromParent();
```

MLIR

```
OpBuilder b(op);                        // insert before op
b.setInsertionPointAfter(op);           // ...or after it
b.setInsertionPointToStart(block);      // ...or at block start
Location loc = op->getLoc();

Value one = arith::ConstantOp::create(b, loc, b.getIndexAttr(1));
Value sum = arith::AddIOp::create(b, loc, x, y);

op->replaceAllUsesWith(newOp);          // redirect all users
op->erase();                            // delete (no uses left!)
```

- LLVM 23 style: `SomeOp::create(builder, loc, …);` older code and docs use `builder.create<SomeOp>(loc, …)`
- Arguments after `loc`: the op's operands, then its attributes (see the op's docs)

MLIR in C++ – Replacing operations

Simple optimization [1]: ⌊(x+y)/2⌋ → (x&y)+((x⊕y)>>1)

```
static void rewriteAvg(arith::DivUIOp div) {
  auto add = div.getLhs().getDefiningOp<arith::AddIOp>();
  APInt c;
  if (!add || !matchPattern(div.getRhs(), m_ConstantInt(&c)) || c != 2)
    return;
  Value x = add.getLhs(), y = add.getRhs();
  OpBuilder b(div);
  Location loc = div.getLoc();
  Value one = arith::ConstantOp::create(
      b, loc, b.getIntegerAttr(x.getType(), 1));
  Value bAnd = arith::AndIOp::create(b, loc, x, y);
  Value bXor = arith::XOrIOp::create(b, loc, x, y);
  Value bShr = arith::ShRUIOp::create(b, loc, bXor, one);
  Value res = arith::AddIOp::create(b, loc, bAnd, bShr);
  div.replaceAllUsesWith(res);
}
```

1. Match `divui(addi(x, y), 2)`

2. Create the new ops in front of `div`:

```
%1 = arith.andi %x, %y : i32
%2 = arith.xori %x, %y : i32
%3 = arith.shrui %2, %c1 : i32
%4 = arith.addi %1, %3 : i32
```

3. Redirect all uses to the new result

[1]: Hacker's delight by Henry S. Warren Jr.

MLIR in C++ – Replacing operations

Let's see if it works!

```
func.func @avg(%x: i32, %y: i32) -> i32 {
  %c2 = arith.constant 2 : i32
  %0 = arith.addi %x, %y : i32
  %1 = arith.divui %0, %c2 : i32
  return %1 : i32
}
```

```
$ mlir-opt avg.mlir \
    --load-pass-plugin=build/src/Avg.so \
    --pass-pipeline="builtin.module(opt-avg)"
```



```
func.func @avg(%arg0: i32, %arg1: i32) -> i32 {
  %c2_i32 = arith.constant 2 : i32
  %0 = arith.addi %arg0, %arg1 : i32
  %c1_i32 = arith.constant 1 : i32
  %1 = arith.andi %arg0, %arg1 : i32
  %2 = arith.xori %arg0, %arg1 : i32
  %3 = arith.shrui %2, %c1_i32 : i32
  %4 = arith.addi %1, %3 : i32
  %5 = arith.divui %0, %c2_i32 : i32
  return %4 : i32
}
```

Dead ops stay behind

Clean up: `builtin.module(opt-avg,canonicalize)` folds constants and removes ops without users

MLIR in C++ – Calling external functions

```
ModuleOp module = getOperation();
OpBuilder b(module.getBodyRegion());   // insert at start of module
auto check = module.lookupSymbol<func::FuncOp>("__runtime_check_div");
if (!check) {
  auto type = b.getFunctionType({b.getI32Type()}, {});
  check = func::FuncOp::create(b, module.getLoc(), "__runtime_check_div", type);
  check.setPrivate();
}
module.walk([&](arith::DivUIOp div) {
  b.setInsertionPoint(div);
  func::CallOp::create(b, div.getLoc(), check, ValueRange{div.getRhs()});
});
```

Look up by name

Declare if missing

Call before every `divui`

```
func.func private @__runtime_check_div(i32)
...
call @__runtime_check_div(%c2_i32) : (i32) -> ()
%1 = arith.divui %0, %c2_i32 : i32
```

The function itself lives in a library you must link against

Useful tips

- Print anything: `llvm::errs() << op`, `op->dump()`, `value.dump()`
- Report problems with the source location: `op->emitError(…)`, `op->emitRemark(…)`
- `--mlir-print-ir-after-all`, `--mlir-print-ir-after-failure` to see what your pass did
- Changing IR while walking it is error-prone: collect ops first, then rewrite (see `runOnOperation` in the task)
- The dialect docs list each op's operands, attributes and regions; the C++ accessors follow their names (`getLhs()`, `getBody()`)

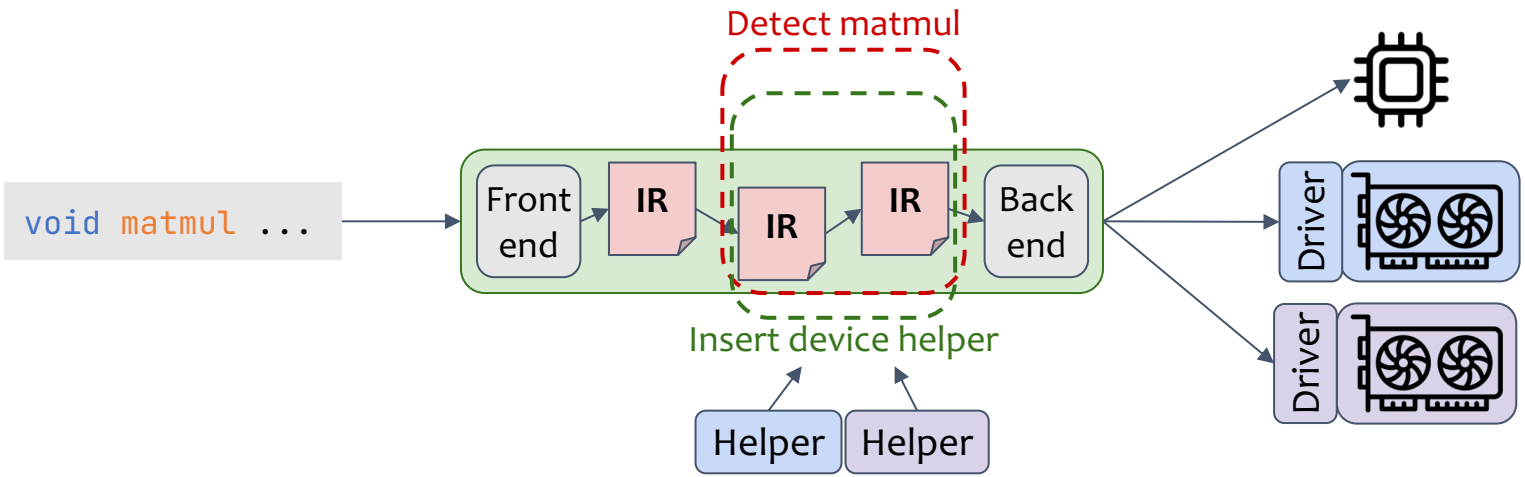
```
llvm::errs() << loop << "\n";          // print an op
loop->dump();                          // same, to stderr
loop.emitRemark("matmul candidate");   // with source location
```

Task 1: Matrix Multiply Optimization in MLIR

Task – Intuition

Problem: How to support automatic offloading?

- User writes a program (matrix multiplication) as normal
- The compiler can decide to optimize it, offload it to an accelerator (GPU, FPGA), …



Task – Detect matmul

How does matmul look in MLIR?

- Three `affine.for`
- A multiplication and addition

```
func.func @matmul(%A: memref<4x3xf32>,
                  %B: memref<3x5xf32>,
                  %C: memref<4x5xf32>) {
  affine.for %i = 0 to 4 {
    affine.for %j = 0 to 5 {
      affine.for %k = 0 to 3 {
        %a = affine.load %A[%i, %k] : memref<4x3xf32>
        %b = affine.load %B[%k, %j] : memref<3x5xf32>
        %p = arith.mulf %a, %b : f32
        %c = affine.load %C[%i, %j] : memref<4x5xf32>
        %s = arith.addf %c, %p : f32
        affine.store %s, %C[%i, %j] : memref<4x5xf32>
      }  }  }
```

Contains only 1 op? (another `affine.for`)

Contains only 1 op? (another `affine.for`)

Contains multiplication and addition?

Exchange with a call to external function

Task – Tiling

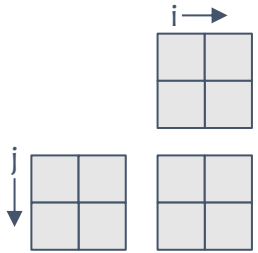
What if the accelerator only accepts fixed-size inputs?

- Source program can have arbitrary (but static) matrix sizes
- Tile input matrix into smaller matmuls

```
func.func @matmul(%A: memref<512x128xf32>,
                  %B: memref<128x512xf32>,
                  %C: memref<512x512xf32>) {
 affine.for %i = 0 to 512 {
  affine.for %j = 0 to 512 {
   affine.for %k = 0 to 128 {
```

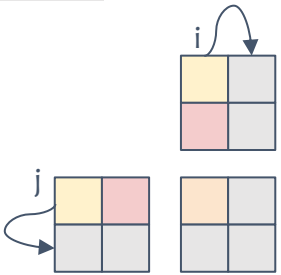
Use subviews to get tiles

Exchange with scf.for over tiles



`affine.for i/j/k`

- step size = 1



`scf.for i/j/k`

- step size = tile size

Resources

MLIR language and dialects

- https://mlir.llvm.org/docs/LangRef/#operations
- https://mlir.llvm.org/docs/Tutorials/UnderstandingTheIRStructure/
- https://mlir.llvm.org/docs/Dialects/Affine/#affinefor-affineaffineforop
- https://mlir.llvm.org/docs/Dialects/MemRef/#memrefsubview-memrefsubviewop
- https://mlir.llvm.org/docs/Dialects/SCFDialect/#scffor-scfforop
- https://mlir.llvm.org/docs/Dialects/ArithOps/
- https://mlir.llvm.org/docs/Dialects/Func/

Passes

- https://mlir.llvm.org/docs/PassManagement/
- https://mlir.llvm.org/docs/Passes/
- https://mlir.llvm.org/docs/Tutorials/Toy/

C++ API (builder pattern: <SomeOp>::create(...))

- https://mlir.llvm.org/doxygen/classmlir_1_1OpBuilder.html
- https://mlir.llvm.org/doxygen/classmlir_1_1Operation.html